

Complete Regex Mastery Guide for SDETs in Java

Table of Contents

1. Introduction to Regex
2. Why Regex is Important for SDETs
3. How Regex Works
4. Regex in Java
5. Exact Matching
6. Metacharacters
7. Character Classes
8. Quantifiers
9. Anchors
10. Escape Characters
11. Predefined Character Classes
12. Groups and Capturing
13. Alternation Operator
14. Greedy vs Lazy Matching
15. Lookaheads
16. Lookbehinds
17. Backreferences
18. Pattern and Matcher Class
19. Common Regex Methods in Java
20. Regex for UI Automation
21. Regex for API Testing
22. Regex for Log Validation
23. Regex for Database Validation
24. Regex for File Validation
25. Regex for Dynamic Data Extraction
26. Regex for Selenium Automation
27. Regex for Playwright Automation
28. Common Real-World SDET Scenarios
29. Regex Optimization Tips
30. Common Beginner Mistakes
31. Advanced Regex Patterns
32. Interview-Oriented Regex Concepts
33. Regex Debugging Strategy
34. Regex Cheat Sheet
35. Final Learning Roadmap

1. Introduction to Regex

Regex stands for:

Regular Expression

A Regex is a pattern used to:

- search text
- validate text
- extract text
- replace text

Regex is heavily used in automation testing.

2. Why Regex is Important for SDETs

Regex is extremely useful in:

Use Case	Example
Form Validation	Email, Password, Mobile Number
API Testing	Token validation
Log Validation	ERROR/WARN logs
Dynamic Data Extraction	OTP, Session ID
File Validation	report.pdf
UI Automation	Dynamic text validation
Data Cleaning	Remove spaces/symbols
Database Validation	Query result validation

3. How Regex Works

Regex works by matching patterns.

Example:

```
\d+
```

Meaning:

- `\d` → digit
- `+` → one or more

Matches:

```
1
22
345
```

4. Regex in Java

Java provides Regex support using:

```
import java.util.regex.*;
```

Main classes:

Class	Purpose
Pattern	Compiles regex
Matcher	Applies regex

5. Exact Matching

Concept

Regex can match exact text.

Example 1

```
String input = "Login";
System.out.println(input.matches("Login"));
```

Output:

```
true
```

Example 2

```
String input = "login";  
System.out.println(input.matches("Login"));
```

Output:

```
false
```

6. Metacharacters

Metacharacters are special symbols with meaning.

Symbol	Meaning
.	Any character
^	Start
\$	End
*	Zero or more
+	One or more
?	Optional
[]	Character class
()	Group

7. Character Classes

Character classes define allowed characters.

Concept 1 — Digits

Regex

```
\d
```

Matches:

0-9

Example 1

```
"5".matches("\\d")
```

Example 2

```
"55".matches("\\d")
```

Returns false because only one digit allowed.

Concept 2 — Alphabets

Regex

```
[A-Za-z]
```

Example 1

```
"A".matches("[A-Za-z]")
```

Example 2

```
"1".matches("[A-Za-z]")
```

Concept 3 — Uppercase Letters

Regex

```
[A-Z]
```

Concept 4 — Lowercase Letters

Regex

[a-z]

8. Quantifiers

Quantifiers define quantity.

8.1 Plus +

Means:

One or more

Regex

\d+

Example 1

```
"123".matches("\\d+")
```

Example 2

```
"abc".matches("\\d+")
```

8.2 Star *

Means:

Zero or more

Regex

```
\d*
```

Example 1

```
"".matches("\\d*")
```

Example 2

```
"123".matches("\\d*")
```

8.3 Question Mark ?

Means:

```
Optional
```

Regex

```
colou?r
```

Example 1

Matches:

```
color
```

Example 2

Matches:

```
colour
```

8.4 Exact Count {n}

Regex

```
\d{10}
```

Exactly 10 digits.

Example 1

```
"9876543210".matches("\\d{10}")
```

Example 2

```
"12345".matches("\\d{10}")
```

8.5 Range {min,max}

Regex

```
\d{2,5}
```

Means:

- minimum 2 digits
 - maximum 5 digits
-

Example 1

```
"123".matches("\\d{2,5}")
```

Example 2

```
"1".matches("\\d{2,5}")
```

9. Anchors

Anchors define position.

9.1 Start Anchor

Regex

```
^A
```

Starts with A.

Example 1

```
"Amit".matches("^A.*")
```

Example 2

```
"Sumit".matches("^A.*")
```

9.2 End Anchor

Regex

```
Z$
```

Ends with Z.

Example 1

```
"TESTZ".matches(".*Z$")
```

Example 2

```
"HELLO".matches(".*Z$")
```

10. Escape Characters

Some characters have special meaning.

To use them literally, escape using `\\`.

Dot Escape

Regex

```
\.
```

Example 1

```
"report.pdf".matches(".*\\..*")
```

Example 2

```
"reportpdf".matches(".*\\..*")
```

11. Predefined Character Classes

Regex	Meaning
<code>\d</code>	digit
<code>\D</code>	non-digit
<code>\w</code>	word character
<code>\W</code>	non-word character
<code>\s</code>	whitespace
<code>\S</code>	non-whitespace

11.1 Word Character `\w`

Regex

```
\w+
```

Matches:

- alphabets
- digits
- underscore

Example 1

```
"amit_123".matches("\\w+")
```

Example 2

```
"amit@123".matches("\\w+")
```

11.2 Whitespace `\s`

Example 1

```
"Hello World".matches(".*\\s.*")
```

Example 2

```
"HelloWorld".matches(".*\\s.*")
```

12. Groups and Capturing

Groups are created using:

()

Groups help:

- extract data
- organize regex
- reuse matches

Example 1 — Extract Error Code

Regex

`ERROR_(\d+)`

Java Example

```
Pattern p = Pattern.compile("ERROR_(\\d+)");
Matcher m = p.matcher("ERROR_500");

if(m.find()) {
    System.out.println(m.group(1));
}
```

Output:

500

Example 2 — Extract Session ID

Regex

`SessionID=(\w+)`

13. Alternation Operator |

Acts like OR.

Example 1 — Browser Validation

Regex

```
(chrome|firefox|edge)
```

Example 1

```
"chrome".matches("(chrome|firefox|edge)")
```

Example 2

```
"opera".matches("(chrome|firefox|edge)")
```

14. Greedy vs Lazy Matching

Very important for extraction.

Greedy Matching

Greedy tries to capture maximum.

Regex

```
<.*>
```

Input

```
<div>Hello</div>
```

Captures everything.

Lazy Matching

Lazy captures minimum.

Regex

```
<.*?>
```

Example 1

Useful in HTML extraction.

Example 2

Useful in XML validation.

15. Lookaheads

Lookaheads validate conditions without consuming characters.

Positive Lookahead

Regex

```
(?=.*\d)
```

Means:

String must contain digit

Example 1 — Password Validation

```
^(?=.*[A-Z])(?=.*\d).\{8,}$
```

Example 2

Checks:

- uppercase
- digit
- minimum length

16. Lookbehinds

Checks what exists before.

Example 1

Regex

```
(?<=₹)\d+
```

Extracts amount after ₹.

Example 2

Input:

```
₹500
```

Output:

```
500
```

17. Backreferences

Backreferences reuse captured groups.

Example 1 — Duplicate Words

Regex

```
\b(\w+)\s+\1\b
```

Input

```
hello hello
```

Example 2 — XML Validation

Regex

```
<([A-Za-z]+)>.*</\1>
```

Ensures opening and closing tags match.

18. Pattern and Matcher Class

Used for advanced regex operations.

Example 1

```
Pattern p = Pattern.compile("\\d+");  
Matcher m = p.matcher("Order123");
```

Example 2

```
while(m.find()) {  
    System.out.println(m.group());  
}
```

19. Common Regex Methods in Java

Method	Purpose
matches()	full match
find()	partial search
group()	extract match
replaceAll()	replace text

Method	Purpose
split()	split text

19.1 matches()

Example 1

```
"123".matches("\\d+")
```

Example 2

```
"abc".matches("\\d+")
```

19.2 find()

Example 1

```
Matcher m = Pattern.compile("\\d+")  
                .matcher("A1 B22");
```

Example 2

```
while(m.find()) {  
    System.out.println(m.group());  
}
```

20. Regex for UI Automation

Commonly used for:

- dynamic labels
 - error messages
 - notifications
 - OTP validation
-

Example 1 — Validate Error Message

```
msg.matches("Error Code: \\d+")
```

Example 2 — Validate Dynamic Welcome Message

```
text.matches("Welcome .*")
```

21. Regex for API Testing

Useful in:

- token validation
- JSON extraction
- response validation

Example 1 — Token Validation

```
^[A-Za-z0-9._-]+$
```

Example 2 — Extract URL

```
https?://[^\s]+
```

22. Regex for Log Validation

Logs are heavily validated using Regex.

Example 1 — Log Level

```
(INFO|WARN|ERROR)
```

Example 2 — Full Log Validation

```
^\d{4}-\d{2}-\d{2}.*$
```

23. Regex for Database Validation

Useful for:

- validating IDs
- validating query outputs
- formatting checks

Example 1 — Employee ID

```
EMP\d{4}
```

Example 2 — Order ID

```
ORD-\d{4}-\d{4}
```

24. Regex for File Validation

Example 1 — PDF Validation

```
.*\.pdf
```

Example 2 — Excel File Validation

```
.*\. (xls|xlsx)
```

25. Regex for Dynamic Data Extraction

Example 1 — OTP Extraction

```
\d{6}
```

Example 2 — Session ID Extraction

```
SessionID=(\w+)
```

26. Regex for Selenium Automation

Useful for:

- validating dynamic texts
 - dynamic element identifiers
 - waits and assertions
-

Example 1

```
Assert.assertTrue(text.matches("Order ID: \d+"));
```

Example 2

```
Assert.assertTrue(msg.matches("Success.*"));
```

27. Regex for Playwright Automation

Useful in assertions.

Example 1

```
expect(locator).toHaveText(Pattern.compile("Welcome .*"));
```

Example 2

```
expect(locator).toContainText("Success");
```

28. Common Real-World SDET Scenarios

Scenario	Regex Usage
OTP Validation	\d{6}
Mobile Number	\d{10}
Email Validation	email regex
API Token	token regex
Logs	ERROR/WARN
Dynamic IDs	capturing groups
HTML/XML	backreferences

29. Regex Optimization Tips

Tip 1

Avoid unnecessary `.*`

Tip 2

Use anchors whenever possible.

Tip 3

Break large regex into smaller parts.

Tip 4

Use groups for readability.

30. Common Beginner Mistakes

Mistake	Problem
Forgetting escape characters	Invalid regex
Missing anchors	Partial matching issues
Overusing <code>.</code>	Slow regex
Memorizing regex	Hard to debug

31. Advanced Regex Patterns

31.1 Strong Password

```
^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@#$$%^&+=!]).{8,}$
```

31.2 IPv4 Validation

```
^((25[0-5]|2[0-4]\d|1\d\d|[1-9]?)\d)\.){3}(25[0-5]|2[0-4]\d|1\d\d|[1-9]?)\d$
```

31.3 XML Tag Pair Validation

```
^<([A-Za-z]+)>.*</\1>$
```

32. Interview-Oriented Regex Concepts

Most asked concepts:

1. Difference between `*` and `+`
2. Difference between `matches()` and `find()`

- 3. Capturing groups
 - 4. Lookaheads
 - 5. Backreferences
 - 6. Greedy vs Lazy
 - 7. `\d`, `\w`, `\s`
 - 8. Anchors
 - 9. Regex optimization
 - 10. Regex in Selenium/API testing
-

33. Regex Debugging Strategy

Best approach:

Step 1

Write valid examples.

Step 2

Write invalid examples.

Step 3

Break regex into smaller parts.

Step 4

Test each part separately.

Step 5

Combine gradually.

34. Regex Cheat Sheet

Regex	Meaning
<code>\d</code>	digit
<code>\D</code>	non-digit

Regex	Meaning
\w	word character
\W	non-word character
\s	whitespace
.	any character
*	zero or more
+	one or more
?	optional
{n}	exact count
{min,max}	range
^	start
\$	end
()	group
[]	character class

35. Final Learning Roadmap

Beginner Level

Learn:

- exact match
- character classes
- quantifiers
- anchors

Practice:

- email
- mobile number
- usernames

Intermediate Level

Learn:

- groups
- alternation

- extraction
- Pattern and Matcher

Practice:

- OTP extraction
 - logs
 - URLs
 - file validation
-

Advanced Level

Learn:

- lookaheads
- lookbehinds
- backreferences
- greedy vs lazy

Practice:

- XML validation
 - strong password
 - API tokens
 - complex logs
-

Final Advice for SDETs

Regex is not about memorizing.

Regex is about:

- understanding patterns
- reading carefully
- practicing real-world cases
- breaking problems logically

The more automation scenarios you solve, the stronger your Regex skill becomes.